

Nova v1.0

*Design and Implementation of a
Multi-Backend Programming Language*

Jivitesh Singh

2 June 2026

Contents

1	Introduction	1
1.1	What is Nova	1
1.2	Motivation and Goals	1
1.3	Overview of Achievements	2
1.4	Paper Structure	2
2	Background	2
2.1	Influences: Go, Python, and Erlang	2
2.2	Compiler Architecture Overview	3
2.3	LLVM as a Backend Target	3
2.4	Related Work	4
3	Language Design	5
3.1	Syntax and Core Semantics	5
3.2	Type System	6
3.3	Memory Model	7
3.4	Concurrency Model	8
4	Implementation	9
4.1	Pipeline Overview	9
4.2	Frontend: Lexer and Parser	10
4.3	Backend 1: Tree-Walking Interpreter	11
4.4	Backend 2: Bytecode Compiler and Virtual Machine	11
4.5	Backend 3: LLVM Native Code Generation	13
4.5.1	IR Generation and the C Runtime	13
4.5.2	Key Challenges	15
4.6	Concurrency Runtime	16
5	Evaluation	17
5.1	Test Suite and Correctness Methodology	17
5.2	VM vs Tree-Walker Performance	17
5.3	Conway’s Game of Life: Serial vs Parallel	18
5.4	Nova vs Go: Parallel Performance Comparison	19

6	Discussion	20
6.1	Key Design Decisions and Trade-offs	20
6.2	What Was Harder Than Expected	21
6.3	Limitations	21
7	Conclusion	22
7.1	Summary of Contributions	22
7.2	Lessons Learned	22
7.3	Future Directions	23
	References	24

1 Introduction

1.1 What is Nova

Nova is a general-purpose programming language implemented from scratch in Rust. It was designed with three priorities: clean, readable syntax with no unnecessary ceremony; a safe concurrency model built into the core of the language; and a complete compiler pipeline that targets native code.

The language draws its primary influences from Go, Python, and Erlang. From Go it takes syntactic simplicity and a goroutine-style concurrency model. From Python it takes expressive, readable syntax and a dynamic type system as the starting point. From Erlang it takes the philosophy of safe, share-nothing message-passing concurrency and structured error handling. The result is a language that feels familiar to developers coming from any of these backgrounds while making different trade-offs at the implementation level.

Nova has three execution backends. The first is a tree-walking interpreter, which evaluates the abstract syntax tree (AST) directly and serves as the reference implementation. The second is a bytecode compiler and virtual machine (VM), which compiles the AST to a flat instruction sequence and executes it with a stack-based call model, achieving 1.4–3× the throughput of the tree-walker on typical workloads. The third is an LLVM-based native code generator, which lowers Nova programs to LLVM Intermediate Representation (IR), links against a hand-written C runtime library, and produces standalone native binaries via `clang`. This backend is the default.

The type system is gradual: type annotations are optional on both function parameters and return types, and unannotated code is never rejected. When annotations are present, a three-pass static checker verifies call sites before execution begins, with type inference filling in return types for unannotated functions automatically. Generics are supported through type variables that are resolved at check time and erased before execution.

1.2 Motivation and Goals

Nova was originally conceived around three pillars: a language that is as easy to use as Go, as safe as Erlang, and capable of self-healing errors powered by AI-assisted diagnostics. While the project demonstrated meaningful progress toward all three, the scope of a full implementation from first principles necessarily narrowed the focus. The goals that were actually delivered, and which this report covers, are:

1. Design and implement a programming language from scratch in Rust, covering the full pipeline from source text to native executable.
2. Build three distinct execution backends, tree-walking interpreter, bytecode virtual machine, and LLVM native code generation, and characterise the performance and trade-offs of each.
3. Implement a gradual static type system with forward-reference support and return-type inference, so that annotated code is checked without requiring annotation everywhere.
4. Build a concurrent runtime with structured parallelism primitives: `spawn/wait`, typed channels, mutex, `select`, `defer`, and timers.
5. Compile to native code via LLVM IR and a hand-written C runtime library, including a post-processing allocation hoisting pass and a tracked allocation pool.
6. Benchmark parallel performance against an equivalent Go implementation on Conway’s Game of Life to assess real-world concurrency overhead.

The honest assessment of the original pitch is addressed in the Discussion (Section 6).

1.3 Overview of Achievements

Nova was developed iteratively over the course of this project. The language supports integers, floats, strings, booleans, nil, arrays, hashmaps, structs, enums, first-class functions, closures, default parameters, destructuring, string interpolation, pattern matching, the pipe operator, and optional semicolons. The standard library provides approximately 60 built-in functions covering string manipulation, array operations, mathematics, type conversion, and I/O.

The type system adds optional static checking without breaking dynamic code. The memory model uses copy-on-write (CoW) semantics backed by reference counting, with a mark-and-sweep cycle collector, for the tree-walking and VM backends, and a tracked allocation pool with a full `nova_gc()` sweep at program exit for the LLVM backend.

The concurrent runtime supports OS-thread-level parallelism with share-nothing semantics: spawned tasks receive deep-cloned copies of their arguments, preventing the most common class of data race at the language level. A rayon-based thread pool, typed channels via the crossbeam library, channel-based mutex primitives, `select` on multiple channels, `defer` for cleanup, and ticker/timeout timers round out the concurrency model.

The LLVM backend compiles Nova to native binaries and includes a type specialisation pass that emits direct LLVM integer and floating-point instructions for statically-known types, bypassing the runtime dispatch cost. Its C runtime library supports the full Nova standard library, including higher-order functions, string operations, and hashmap operations, enabling all three backends to run the same benchmark suite.

On the Conway’s Game of Life benchmark (512×512 grid, 1000 generations, 4 threads), Nova achieves a 2.02× parallel speedup, compared to 2.17× for the equivalent Go implementation. The raw serial throughput is approximately 25× slower than Go, a gap fully explained by Nova’s dynamic tagged-value representation versus Go’s native byte arrays.

1.4 Paper Structure

Section 2 surveys the languages that influenced Nova’s design and introduces compiler architecture concepts. Section 3 describes the language design: syntax, type system, memory model, and concurrency model. Section 4 covers the implementation in detail across all three backends. Section 5 presents the test methodology and benchmark results. Section 6 reflects on design decisions, unexpected difficulties, and limitations. Section 7 concludes.

2 Background

2.1 Influences: Go, Python, and Erlang

Three languages had the most direct influence on Nova’s design.

Go established the template for what “simplicity at scale” looks like. Its goroutine model, lightweight concurrent tasks scheduled over a thread pool, is the direct ancestor of Nova’s `spawn/wait` primitives. Go also demonstrated that a language without generics for many years could still be widely adopted, and that a simple type system with fast compilation could outcompete more complex alternatives for practical software development. Nova borrows Go’s philosophy of keeping the language core small and putting complexity in the standard library rather than the type system. However, Go’s verbose error handling (`if err != nil`) and lack of pattern matching, string interpolation, and closures with captured mutable state are areas where Nova deliberately diverges.

Python provided the syntactic model. Python demonstrated that readable, low-ceremony syntax, no type declarations required, no semicolons, indented blocks, first-class functions, can serve both beginners writing their first program and

professionals writing production systems. Nova takes Python’s aesthetic but replaces indentation-sensitivity with explicit curly braces (removing the whitespace ambiguity that can affect Python tooling) and adds a gradual type layer that Python only acquired incrementally through PEP 484 and later.

Erlang provided the concurrency philosophy. Erlang pioneered the actor model at the language level: each process has its own heap, processes communicate only through message passing, and a crash in one process cannot corrupt another. Nova’s deep-clone-on-spawn mechanism is a direct attempt to enforce this same principle at the API level: when a task is spawned, it receives a fully independent copy of its arguments, not a shared reference. This prevents the most common class of Go-style data race, where a goroutine captures a reference to a variable that the parent also mutates after the spawn. The Discussion (Section 6) examines how far this mechanism goes toward Erlang-style safety, and where the gap remains.

2.2 Compiler Architecture Overview

A compiler translates a program from source text to an executable form. The architecture choices made at this level determine the language’s performance ceiling, debugging experience, and implementation complexity. Three architectures are relevant to Nova.

A **tree-walking interpreter** evaluates the program by recursively walking the abstract syntax tree produced by the parser. Each node is a function call in the implementation language. This is the simplest architecture to implement correctly and is the standard choice for a first version of a language. The cost is performance: every operation requires pointer indirection through tree nodes, and function call dispatch is expensive because the interpreter cannot predict which node type it will encounter next. The tree-walker also inherits the implementation language’s call stack for the interpreted program’s call stack, which limits recursion depth. Nova’s tree-walking interpreter, implemented in `evaluator.rs`, serves as the reference implementation and was frozen once the bytecode VM reached full feature parity and became the default backend. At that point, maintaining three diverging implementations of every new language feature would have multiplied the implementation cost without adding research value. All subsequent features, structs, enums, methods, bitwise operators, and the full concurrency runtime, were developed on the VM and LLVM backends only.

A **bytecode virtual machine** inserts a compilation step between parsing and execution. The AST is compiled to a flat array of simple instructions (bytecode). A dispatch loop then executes instructions one at a time, maintaining an operand stack and a call stack of frames. The flat instruction array has dramatically better CPU cache behaviour than a tree, since sequential instructions are adjacent in memory and can be prefetched. Function calls are handled by pushing and popping stack frames rather than by Rust recursion, removing the recursion depth limit. Nova’s VM achieves 1.4–3× the throughput of the tree-walker on compute and higher-order-function workloads.

A **native code compiler** produces machine code that runs directly on the CPU with no interpreter overhead. Rather than writing a backend from scratch, a multi-year project covering register allocation, instruction selection, and binary format, Nova uses LLVM as the backend. Nova’s code generator (`codegen.rs`) translates the AST to LLVM IR, a well-defined text-format intermediate representation. LLVM then handles optimisation (constant folding, dead code elimination, loop unrolling), instruction selection, and binary generation. The resulting binary links against a hand-written C runtime library (`nova_rt.c`) that implements all dynamic operations.

2.3 LLVM as a Backend Target

LLVM IR is a typed, Static Single Assignment (SSA) form intermediate representation (Lattner and Adve, 2004). Every value is assigned exactly once, and control flow is represented as a directed graph of basic blocks connected by branch instructions. This form is amenable to classical optimisations: use-def chains are explicit, and transformations like `mem2reg`

(promoting stack allocations to SSA values) and scalar replacement of aggregates (SROA) can be applied without alias analysis.

For Nova, LLVM was the right choice for three reasons. First, it produces competitive native code without requiring Nova to implement a register allocator or instruction scheduler. Second, it handles platform differences (x86, ARM, Windows, Linux) through target triples, meaning the same LLVM IR compiles to native binaries on any supported platform. Third, the IR format is readable text, which made debugging the code generator tractable: a wrong value always produces wrong IR text that can be inspected directly.

Nova’s approach to LLVM IR is conservative by design. Every Nova value is represented as a tagged union struct:

```
1 typedef struct {
2     uint64_t tag;          /* 0=nil, 1=bool, 2=int, 3=float,
3                             4=str, 5=array, 6=map, 7=closure,
4                             8=enum, 9=task, 10=chan, 11=mutex */
5     uint64_t payload; /* int/bool: stored directly;
6                       float: bit pattern via memcpy;
7                       pointers: cast to uint64_t */
8 } NovaValue;
```

Listing 1: NovaValue tagged union in nova_rt.c

All operations, arithmetic, comparisons, and indexing, are implemented as C functions in `nova_rt.c` that take and return `NovaValue` pointers. The code generator calls these functions by emitting `call` instructions in LLVM IR. Type specialisation (Section 4.5) bypasses these calls for statically-known integer and float expressions by emitting direct LLVM arithmetic instructions.

2.4 Related Work

The most directly comparable project is Lox, the language implemented in *Crafting Interpreters* (Nystrom, 2021), which also builds a tree-walker and a bytecode VM for a dynamically-typed language. Nova differs in scope: it adds a third LLVM backend, a concurrent runtime, and a gradual type system, none of which are present in Lox.

Gradual typing as a formal system was introduced by Siek and Taha (2006) and is now embodied in TypeScript, Python’s mypy, and Typed Racket. Nova’s type checker is less complete than these systems, as it does not implement blame tracking or contract checking, but follows the same core principle: unannotated code is compatible with any type, and the checker only fires when it has enough information to be sure of a violation.

Languages that have taken the LLVM backend path include Clang (C/C++ front end for LLVM Clang), Kotlin/Native, and Swift. Nova’s approach is closer to a teaching implementation than a production compiler: the IR is emitted as text strings, not through the LLVM C++ API, which trades correctness guarantees for simplicity.

The share-nothing spawn model is directly inspired by Erlang and is also present in Pony, which uses capabilities to enforce reference isolation at the type level. Nova’s enforcement is at the API level (deep clone on spawn) rather than the type level, which is weaker but far simpler to implement.

3 Language Design

3.1 Syntax and Core Semantics

Nova's syntax is designed around the principle that common operations should be concise and uncommon operations should be explicit. The language has no required semicolons (they are accepted as optional statement terminators to resolve a parsing ambiguity described in Section 4.2), uses curly braces for block delimiters, and requires no boilerplate for simple programs.

A Nova program demonstrating the core syntax is shown below.

```
1 fn fib(n: int) -> int {  
2     if n <= 1 { return n }  
3     fib(n - 1) + fib(n - 2)  
4 }  
5  
6 let names = ["alice", "bob", "carol"]  
7 for name in names {  
8     print "hello {name}"  
9 }  
10  
11 let result = match fib(10) {  
12     55 => "correct"  
13     -  => "unexpected"  
14 }
```

Listing 2: Core Nova syntax

Several design choices are worth noting.

Implicit return. A function returns the value of its last expression without a `return` keyword. Explicit `return` is also supported for early exits. This follows Rust and Ruby's convention and reduces boilerplate for short functions.

String interpolation. `"hello {name}"` is resolved at runtime by re-parsing the expression inside the braces and converting the result to a string. The expression inside `{...}` can be any valid Nova expression, including function calls and arithmetic.

Pattern matching. The `match` expression dispatches on exact values, range patterns (`1..10`), and enum variants. It is an expression, not a statement: the value of the matched arm is the value of the whole `match`.

Pipe operator. `x |> f(extra)` desugars to `f(x, extra)` at parse time. This allows linear data transformation pipelines without deeply nested function calls.

```
1 let result = [1, 2, 3, 4, 5]  
2     |> filter((x) -> x > 2)  
3     |> map((x) -> x * x)  
4     |> sum()           // result = 50
```

Listing 3: Pipe operator and higher-order functions

First-class functions and closures. Functions are values. Lambdas use the `(params) -> expr` or `(params) -> { body }` syntax and capture the enclosing scope at creation time. Named functions capture globals but not locals of their enclosing scope, consistent with the memory model described in Section 3.3.

Structs, enums, and methods. User-defined types are supported through `struct` definitions, `enum` tagged unions, and `impl` blocks that attach methods to a type. `self` is an explicit first parameter of every method, following Rust’s convention.

```
1 struct Point { x, y }
2
3 impl Point {
4     fn distance(self) -> float {
5         sqrt(self.x * self.x + self.y * self.y)
6     }
7     fn translate(self, dx, dy) {
8         self.x += dx
9         self.y += dy
10    }
11 }
12
13 let p = Point { x: 3.0, y: 4.0 }
14 print p.distance()    // 5.0
```

Listing 4: Struct and method definition

3.2 Type System

Nova’s type system is gradual (Siek and Taha, 2006): type annotations are entirely optional, and unannotated code is always accepted. The checker only produces errors when it can prove a mismatch from the information it has.

Annotations can be applied to function parameters and return types using either `: type` or `-> type` syntax:

```
1 fn add(a: int, b: int) -> int {
2     a + b
3 }
4
5 fn identity<T>(x: T) -> T {
6     x
7 }
```

Listing 5: Type annotations

The type checker operates in three passes over the AST, all running before execution begins.

Pass 1 — Signature collection. Every function declaration is scanned and its parameter types and return type annotation (if present) are stored in a signature map. This pass makes forward references work: a function called before it is defined is still type-checked because the checker has already seen the callee’s signature.

Pass 2 — Return type inference. For functions without an explicit return type annotation, the checker infers the return type from the function body. It collects all explicit `return` statements and the implicit last expression. If all paths produce the same type, that type is recorded. If they disagree, the function is left as **Unknown**, producing no false errors on genuinely dynamic code. This pass is run as a fixed-point loop (up to ten iterations) so that chains of unannotated functions calling each other resolve in the correct order regardless of declaration position.

For example, given:

```

1 fn double(x: int) -> int { x * 2 }
2 fn wrap(x) { double(x) }

```

`wrap` has no return annotation. In pass 1 only `double`’s signature (`int -> int`) is recorded; `wrap`’s return type is `Unknown`. In the first iteration of the fixed-point loop, the checker analyses `wrap`’s body: the call `double(x)` has a known return type of `int`, so `wrap`’s return type is inferred as `int` and recorded. A second iteration confirms no further changes. The loop terminates and pass 3 can now type-check calls to `wrap` correctly, even though `wrap` carries no annotation in the source.

Pass 3 — Call-site checking. Every function call is visited. Each argument type is inferred and compared to the corresponding parameter annotation. For generic functions, type variables are resolved from the actual argument types before checking.

The type hierarchy has two widening rules. An `int` value is compatible anywhere a `float` is expected, matching the language’s arithmetic promotion rules. An untyped `array` is compatible with any typed array, so an empty `[]` literal is never rejected when passed to a function expecting `[int]`.

Generics are erased at runtime. Type variables (`T`, `U`, etc.) exist only in the checker; the evaluator, VM, and LLVM backend never see them.

3.3 Memory Model

The tree-walker and VM backends share the same memory model, implemented in `evaluator.rs`.

Reference counting with copy-on-write. Arrays and hashmaps are heap-allocated and managed by Rust’s `Arc<Mutex<T>>`: multiple variables can share the same backing store, and cloning is $O(1)$ (just an atomic reference count increment). When a variable is assigned from another (`let b = a`), both share the same allocation.

Mutation triggers a copy-on-write check. Before any write to an array element or hashmap key, the runtime checks `Arc::strong_count`: if it is greater than 1 (meaning other variables share the same data), the inner data is cloned into a fresh allocation before the write. The result is value semantics at the language level with near-reference performance for read-heavy workloads.

```

1 let a = [1, 2, 3]
2 let b = a           // O(1) -- b and a share backing store
3 b[0] = 99           // CoW triggers: b gets a private copy
4 print a[0]          // prints 1 -- a is unaffected

```

Listing 6: Copy-on-write semantics

Cycle collection. Reference counting alone cannot free cycles (two objects that point to each other). A mark-and-sweep collector runs after each top-level statement in both the tree-walker and the bytecode VM. It walks the environment from the root, marks every reachable `Arc` by raw pointer address, and clears the contents of any registered allocation that was not marked. In practice, Nova’s CoW semantics prevent cycles from forming in the first place: any self-referential assignment triggers a private copy at the point of assignment, so the resulting structure is always a directed acyclic graph. The collector exists as a correct safety net; in practice `cycles_collected` stays at zero for normal programs.

LLVM memory model. The LLVM backend uses `malloc` for all heap allocations (strings, arrays, maps, closures, enums) routed through a tracked pool in `nova_rt.c`. A `nova_gc()` call at program exit frees every tracked pointer. There is no runtime cycle detection or reference counting in the LLVM backend. For short-lived programs this is sufficient; for long-running programs, the pool grows monotonically until exit. Full GC in the LLVM backend is left as future work.

One correctness detail worth noting: Nova’s `sort()` builtin uses `total_cmp` rather than `partial_cmp` for floating-point comparisons. `partial_cmp` returns `None` for NaN (since NaN has no defined ordering), and calling `unwrap()` on that `None` would panic at runtime. `total_cmp` imposes a deterministic total ordering on all floating-point values including NaN and infinities, so sorting an array that contains NaN produces a consistent result without crashing.

3.4 Concurrency Model

Nova’s concurrency model is built around OS threads with share-nothing semantics.

spawn and wait. `spawn expr` runs `expr` on a separate thread and returns a task handle. `wait(t)` blocks until the task completes and returns its value. If the task threw an uncaught error, `wait` re-throws it in the parent thread.

```
1 fn compute(n: int) -> int { fib(n) }
2
3 let t1 = spawn compute(30)
4 let t2 = spawn compute(28)
5 print wait(t1) + wait(t2)
```

Listing 7: Basic spawn/wait

Share-nothing by default. When a task is spawned, its arguments are deep-cloned: a fully independent copy with no shared `Arc` pointers is made and handed to the new thread. This prevents the canonical data race of Go-style goroutines, where a closure capturing a slice reference can race with the parent that also holds the slice. In Nova, the parent and the spawned task always start with independent data.

Channels. `channel()` creates an unbounded channel backed by crossbeam’s multi-producer multi-consumer (MPMC) queue. `send(ch, val)` deep-clones the value and sends it. `recv(ch)` blocks until a value is available. `tryRecv(ch)` returns `nil` immediately if the channel is empty.

Mutex. `mutex(val)` creates a mutex pre-loaded with a protected value. Internally it uses a bounded(1) crossbeam channel as a binary semaphore: the channel capacity enforces mutual exclusion. `lock(m)` removes the token (blocking until it is available), `unlock(m, new_val)` returns a new value. `withLock(m, fn)` atomically takes the lock, calls `fn(current_val)`, and stores the return value.

Select. `select { case ch -> v { body } ... }` blocks until one of the listed channels has a value, then executes the corresponding arm. An optional `default` arm makes the select non-blocking.

Defer. `defer expr` schedules `expr` to run when the enclosing function returns, in last-in, first-out (LIFO) order. This is the standard mechanism for cleanup operations and is guaranteed to run even on early return. In the tree-walker and VM, each active function frame maintains a stack of deferred expressions; on `Return`, the stack is drained in reverse order before the frame is popped. In the LLVM backend, each `CallFrame` holds a `Vec<Arc<Chunk>>` of deferred sub-chunks that are executed by the VM loop at the function exit point.

Timers. `ticker(ms)` returns a channel that fires every `ms` milliseconds on a background OS thread. `timeout(ms)` returns a channel that fires once after `ms` milliseconds. Both stop automatically when the channel is garbage collected.

Thread scheduling. Under the LLVM and VM backends, `spawn` uses a rayon work-stealing thread pool (default size: one thread per logical CPU core). Ticker and timeout use dedicated OS threads, since they spend most of their time blocked in `sleep` and would waste a compute worker.

4 Implementation

4.1 Pipeline Overview

The pipeline has a common front end (lexer, parser, warnings, type checker) followed by three back ends that share no code.

```
Source code
|
v
lexer.rs      --> Vec<(Token, usize)>
|
v
parser.rs     --> Vec<Expr>
|
v
warnings.rs   --> unused variable warnings
|
v
typechecker.rs --> static type errors (exit 1 on failure)
|
|-- nova run (default) / nova build -----> codegen.rs
|                                           AST --> LLVM IR
|                                           hoist_allocas pass
|                                           clang + nova_rt.o
|                                           --> native binary
|
|-- nova run --vm -----> compiler.rs
|                                           AST --> Vec<Instruction>
|                                           vm.rs (ip loop)
|
'-- nova run --tree -----> evaluator.rs
                                           recursive eval()
```

The `warnings.rs` module runs a two-pass walk over the AST before any backend executes. The first pass collects every variable name declared with `let`, recording its source line. The second pass collects every variable name that appears in a read position anywhere in the program. Any name that was declared but never read produces an unused-variable warning printed to `stderr`. The check is suppressed in REPL mode because the user types line by line and the checker cannot see future lines, so every `let` would produce a spurious warning.

The `error.rs` module is a shared utility used by all three backends: it implements `nova_error()`, which stores the error message in a thread-local `LAST_ERROR`, then panics with a sentinel string that `catch_unwind` can intercept in the read-eval-print loop (REPL) and in spawned threads. Error messages include a Damerau-Levenshtein distance check against all names in scope: if the undefined name is within edit distance 2 of a known name (accounting for transpositions as well as insertions, deletions, and substitutions), Nova appends a “Did you mean x?” suggestion to the error.

The `repl.rs` module implements the interactive Read-Evaluate-Print Loop (REPL). Each input block is wrapped in `catch_unwind(AssertUnwindSafe(...))` so that a runtime error recovers without terminating the session. Before evaluating each block, the REPL snapshots the current environment; on error, the snapshot is restored so that broken state (a partially-constructed variable, a failed assignment) does not persist into the next prompt. Non-nil expression results

are printed automatically, and multi-line blocks are detected by counting unmatched braces so the prompt switches from `nova>` to `...>` during continuation.

4.2 Frontend: Lexer and Parser

Lexer. The lexer (`lexer.rs`) converts raw source text to a `Vec<(Token, usize)>`, where each token is paired with its source line number. The source is stored as `Vec<char>` rather than as a raw `String`: indexing into a Rust `String` by character position is $O(n)$ because strings are UTF-8 encoded and characters are variable width. The `Vec<char>` representation makes character-level access $O(1)$ and avoids multi-byte boundary bugs. Line numbers are assigned during scanning so that all subsequent pipeline stages, warnings, type checker, and runtime errors, can report accurate source locations without re-scanning.

Parser. The parser (`parser.rs`) is a hand-written recursive descent parser that produces a `Vec<Expr>`, where `Expr` is a Rust enum representing every possible AST node. Recursive descent was chosen over a table-driven approach (such as look-ahead LR (LALR)) because it is straightforward to debug, produces good error messages, and can handle context-sensitive constructs (such as distinguishing a lambda `(x) -> x+1` from a grouped expression `(x+1)`) with a short lookahead.

Operator precedence. Recursive descent enforces operator precedence through the call chain structure: lower-precedence operators sit at the top of the chain, and each function calls the next-higher-precedence function for its sub-expressions. This means higher-priority operators are resolved by deeper (inner) calls, and lower-priority operators are resolved by shallower (outer) calls.

Nova's full precedence chain, from lowest to highest, is:

```
parse_null_coalesce → parse_or → parse_and → parse_pipe → parse_comparison → parse_bitor →  
  parse_bitxor → parse_bitand → parse_expr (+, -) → parse_term (*, /, %) → parse_primary
```

The key insight is that the call stack enforces precedence: deeper calls bind more tightly. To see why, consider `1 + 2 * 3`:

1. `parse_expr` is called. It calls `parse_term` for the left operand, which returns 1.
2. `parse_expr` sees `+`, so it calls `parse_term` for the right operand.
3. `parse_term` calls `parse_primary` for 2, sees `*`, calls `parse_primary` for 3, and returns `(2 * 3)`.
4. `parse_expr` combines: `1 + (2 * 3) = 7`.

If the precedence were reversed (`+` higher than `*`), `parse_term` would call `parse_expr`, the `+` would be resolved first, and the result would be `(1 + 2) * 3 = 9`. The call depth is the precedence level.

AST representation. The `Expr` enum in `parser.rs` covers approximately 45 node types, from simple literals (`Expr::IntLit(i64)`) to complex constructs (`Expr::Match`, `Expr::Select`). Recursive nodes use `Box<Expr>` to break the recursion and give Rust a fixed size at compile time. Every statement produced by `parse_statement` is wrapped in `Expr::Line(usize, Box<Expr>)` to track its source line, so line numbers propagate through all three backends without adding a field to every variant.

4.3 Backend 1: Tree-Walking Interpreter

The tree-walking interpreter (`evaluator.rs`) implements a single recursive `eval(expr, env)` function that dispatches on the `Expr` variant and computes a `Value`. The environment is a `HashMap<String, Value>` that maps variable names to their current values.

Function calls clone the environment into a new frame (`fn_env = env.clone()`), evaluate the body within that frame, then write back any keys that existed in the outer scope before the call (excluding parameter names, which shadow outer bindings). This two-step clone-and-writeback mechanism gives functions access to global state while keeping local declarations isolated: a `let x` inside a function does not overwrite an outer `x`.

The interpreter supports the full language including closures (which snapshot the environment at creation time and store it in a shared `Arc<Mutex<Env>>` to support mutable state across calls), mark-and-sweep cycle collection, and all concurrency primitives. It is frozen; all subsequent language features were developed on the VM and LLVM backends.

Recursion depth is capped at 500 via a `CALL_DEPTH` thread-local counter that is incremented on every function call and decremented on return. Exceeding the limit throws a clean Nova error rather than letting the OS stack overflow and crash the process silently. The VM has no such cap because its call stack is a heap-allocated `Vec<CallFrame>` rather than the Rust call stack, so it is limited only by available memory.

The `--memory` flag (`nova run --tree --memory file.nova`) prints a memory report to `stderr` after the program exits:

```
--- memory report ---
arrays allocated:    512
hashmaps allocated:  10
cycles collected:    0
live arrays:         3
live hashmaps:       1
peak live objects:   48
```

The report is produced by reading the allocation counters incremented by `make_array` and `make_map`, and scanning the `ARRAY_HEAP` and `MAP_HEAP` registries for entries whose `Weak` pointers are still upgradeable. It is only available with `--tree` because the VM and LLVM backends do not share the same allocation registry.

4.4 Backend 2: Bytecode Compiler and Virtual Machine

The bytecode backend consists of a compiler (`compiler.rs`) that lowers the AST to a flat `Vec<Instruction>` (called a `Chunk`), and a virtual machine (`vm.rs`) that executes that chunk.

Compiler. The compiler makes a single pass over the AST. Expressions push a value onto an operand stack; statements consume values and produce side effects. Forward jumps (for `if` branches and loop back-edges) are handled by back-patching: a placeholder instruction is emitted with target index 0, and the real target is filled in once the target position is known. Break and continue use a `LoopFrame` stack that accumulates placeholder indices to be backpatched when the enclosing loop ends.

For example, `if cond { A } else { B }` compiles as follows:

```
0  <compile cond>
1  JumpIfFalse(0)      -- placeholder; target unknown
2  <compile A>
3  Jump(0)             -- placeholder; skip else
4  <compile B>
```

After A is compiled the real target for the `JumpIfFalse` is known (index 4, the start of B), so instruction 1 is overwritten: `chunk.patch(1, JumpIfFalse(4))`. After B is compiled the `Jump` placeholder at index 3 is patched to the instruction immediately past B. The entire construct is compiled in a single forward pass with no lookahead.

VM. The VM maintains an instruction pointer (`ip`) and a call stack of `CallFrames`. Each frame contains a reference to its `Chunk`, the current `ip`, and a `HashMap` of local variables. The main dispatch loop clones the current instruction before matching it, which drops the immutable borrow on the frame before any mutable operation such as pushing a new frame, which satisfies Rust's borrow checker without unsafe code.

```
loop {
    let ip = frames.last().unwrap().ip;
    let instr = frames.last().unwrap().chunk.code[ip].clone();
    frames.last_mut().unwrap().ip += 1;

    match instr {
        Instruction::LoadConst(v) => stack.push(v),
        Instruction::Add           => { let b = stack.pop(); let a = stack.pop();
                                     stack.push(add(a, b)); },
        Instruction::Call(name, argc) => { /* push CallFrame */ },
        Instruction::Return          => { /* pop CallFrame, push result */ },
        ...
    }
}
```

Listing 8: VM dispatch loop (simplified)

To make this concrete, consider the statement `let x = 1 + 2`. The compiler produces four instructions:

```
0 LoadConst(Int(1))
1 LoadConst(Int(2))
2 Add
3 DefineGlobal("x")
```

The VM executes them in order, maintaining a value stack:

```
ip=0 LoadConst(Int(1))    stack: [1]
ip=1 LoadConst(Int(2))    stack: [1, 2]
ip=2 Add                  stack: [3]      -- pops 1 and 2, pushes 3
ip=3 DefineGlobal("x")     stack: []      -- pops 3, inserts x=3 into globals
```

Instructions communicate only through the stack. `Add` has no knowledge of 1 or 2 specifically; it simply pops the top two values and pushes their sum. The compiler is responsible for ensuring the correct values are on the stack in the correct order when each instruction executes.

Function calls extend this with a second dimension: the call stack of `CallFrames`. For `fn add(a, b) { a + b }` called as `add(1, 2)`, the compiler produces two chunks. The top-level chunk calls the function:

```
-- top-level chunk
0 LoadConst(Int(1))
1 LoadConst(Int(2))
2 Call("add", 2)      -- pops 2 args, pushes new CallFrame
3 DefineGlobal("result")
```

The `add` function body is a separate chunk:

```
-- add chunk
0  LoadLocal("a")
1  LoadLocal("b")
2  Add
3  Return          -- pops frame, pushes result onto caller's stack
```

The VM execution trace:

```
[top frame] ip=0  LoadConst(1)      stack: [1]
[top frame] ip=1  LoadConst(2)      stack: [1, 2]
[top frame] ip=2  Call("add", 2)    -- pops 1 and 2, pushes CallFrame
                                   locals: {a:1, b:2}, stack: []
[add frame] ip=0  LoadLocal("a")    stack: [1]
[add frame] ip=1  LoadLocal("b")    stack: [1, 2]
[add frame] ip=2  Add                stack: [3]
[add frame] ip=3  Return              -- pops add frame, pushes 3 onto top frame
[top frame] ip=3  DefineGlobal       stack: []  globals: {result: 3}
```

Each frame has its own `ip` and `locals` map; the value stack is shared across all frames, which is how the return value transfers from the callee back to the caller.

Global variables are stored in a flat `HashMap` shared across all frames, which fixes the tree-walker's scoping limitation where global mutations inside functions were silently discarded. All array and hashmap allocations go through the same `make_array/make_map` registration path as the tree-walker, so the shared `collect_cycles` function runs after every top-level `Pop` instruction and provides the same cycle-collection safety net as the tree-walking backend.

4.5 Backend 3: LLVM Native Code Generation

4.5.1 IR Generation and the C Runtime

The LLVM backend consists of two components: the code generator `codegen.rs`, which translates the Nova AST to LLVM IR text, and the C runtime library `nova_rt.c`, which implements all dynamic operations.

Every Nova value (integer, string, array, closure, nil) is represented at the C level as a `NovaValue`: a 16-byte tagged union of two `i64` fields.

```
1 typedef struct { uint64_t tag; uint64_t payload; } NovaValue;
```

The `tag` field identifies the type at runtime:

Tag	Type
0	nil
1	bool (payload: 0 or 1)
2	int (payload: i64 bit pattern)
3	float (payload: IEEE-754 double reinterpreted as i64)
4	string (payload: pointer to null-terminated C string)
5	array (payload: pointer to heap-allocated <code>NovaArray</code>)
6	map (payload: pointer to heap-allocated <code>NovaMap</code>)
7	closure (payload: pointer to heap-allocated <code>NovaClosure</code>)
8	enum variant (payload: pointer to heap-allocated <code>NovaEnum</code>)
9	task (payload: pointer to heap-allocated <code>NovaTask</code>)
10	channel (payload: pointer to heap-allocated <code>NovaChan</code>)

Every Nova variable in the LLVM backend is an `alloca %NovaValue`, a 16-byte slot on the stack frame. Runtime functions always receive and write values through pointers to these slots; no `NovaValue` is ever returned by value, which avoids struct-return Application Binary Interface (ABI) differences between platforms.

The split exists because LLVM IR has no concept of a dynamically-typed tagged value. Rather than re-implementing all of Nova’s runtime logic in IR (which would be correct but unreadable), all runtime operations are written as plain C functions in `nova_rt.c`. The code generator only needs to know each function’s signature to emit a `call` instruction; the actual behaviour lives in C. A representative set of runtime functions:

```

1 void nova_add(const NovaValue* a, const NovaValue* b, NovaValue* out);
2 void nova_index_get(const NovaValue* arr, const NovaValue* idx, NovaValue* out);
3 void nova_index_set(NovaValue* arr, const NovaValue* idx, const NovaValue* val);
4 void nova_make_closure(ClosureDispatch fn, int64_t env_size,
5                        const NovaValue* env, NovaValue* out);
6 void nova_spawn(const NovaValue* closure, NovaValue* out);
7 void nova_gc(void);

```

Listing 9: Selected `nova_rt.c` signatures

All functions pass values by pointer to avoid struct-return application binary interface (ABI) differences between Clang and LLVM IR. The code generator allocates a `%NovaValue` slot for every sub-expression result (`%t0`, `%t1`, ...), emits a `call` into the appropriate runtime function, and passes the output slot to the next instruction.

Before compiling any IR, `compile_program` runs a `prescan_stmts` pass over the AST that registers every function name into `defined_fns`. This makes forward references work: a function called before it is defined in the source is still compiled correctly because the codegen already knows the name is a user-defined function and emits a `@nova_fn.X` call rather than treating it as an unknown builtin.

String literals are interned as LLVM global constants at the top of the `.ll` file. Every string encountered during compilation is assigned a unique global name (`@str0`, `@str1`, ...) and emitted as a null-terminated byte array:

```
@str0 = private constant [6 x i8] c"hello\00"
```

Non-printable characters and special bytes are converted to `\XY` hex escapes by the `escape_for_llvm` pass so the IR text is always valid ASCII. At runtime, the `nova_make_str` call receives a pointer directly to this read-only global, so no heap allocation is needed for string literals.

A complete LLVM IR fragment for the expression `a + b` where both are integers:

```

1 %t0 = alloca %NovaValue, align 8
2 call void @nova_add(ptr %local_a, ptr %local_b, ptr %t0)

```

Listing 10: LLVM IR emitted for $a + b$

When both `a` and `b` are statically known to be integers (through the type inference described in Section 3.2), the type specialisation pass replaces the `nova_add` call with direct LLVM integer arithmetic:

```

1 %raw_a = getelementptr %NovaValue, ptr %local_a, i64 0, i32 1
2 %val_a = load i64, ptr %raw_a, align 8
3 %raw_b = getelementptr %NovaValue, ptr %local_b, i64 0, i32 1
4 %val_b = load i64, ptr %raw_b, align 8
5 %result = add i64 %val_a, %val_b
6 ; store back: write tag=2 to field 0, write %result to field 1

```

Listing 11: Specialised IR for `int + int`

This eliminates the function call overhead for tight loops over integer data.

The `nova_rt.o` object file is compiled once from `nova_rt.c` using `clang`. Both `nova run` and `nova build` check whether `nova_rt.o` is missing or older than `nova_rt.c` and rebuild it automatically, so users never need to invoke `clang` manually.

The two user-facing compilation commands differ in what they do with the output:

`nova run file.nova` (the default) writes the `.ll` file and the compiled binary to the system temp directory, runs the binary immediately, then deletes both, leaving no build artefacts on disk. This mirrors `go run`: a single command compiles and runs without producing permanent files.

`nova build file.nova` writes `file.ll` and `file.exe` next to the source file and keeps them. The resulting binary is fully self-contained: it links `nova_rt.o` statically and depends only on the C runtime, so it can be distributed and run on any compatible machine without Nova installed.

4.5.2 Key Challenges

Alloca hoisting and stack overflow. LLVM’s `mem2reg` pass, which promotes stack allocations to SSA registers, only operates on `alloca` instructions in the **entry block** of a function. Nova’s code generator initially emitted `alloca %NovaValue, align 8` instructions inside loop bodies (non-entry basic blocks). Each loop iteration therefore executed a `sub rsp, 16` instruction at the machine level to grow the stack frame for the new allocation. For the 512×512 GoL grid with approximately 20 temporaries per cell, this produced $512 \times 512 \times 20 \times 16 \approx 80$ MB of stack growth before any output was produced. The Windows default thread stack is 1 MB, causing a silent process crash before `main` could print anything.

The fix is a post-processing pass, `hoist_allocas`, added after IR generation in `compile_program`. The pass scans the emitted IR line by line, collects all `= alloca %NovaValue, align 8` instructions from non-entry blocks, removes them from their original location, and inserts them at the top of the function entry block. Environment arrays (`alloca [N x %NovaValue]`) are intentionally not hoisted, as they must be reinitialised each iteration and are immediately copied to the heap by `nova_make_closure`.

Integer division type bug. The second GoL bug was more subtle. The parallel workers ran without errors but produced an incorrect alive count identical to the initial seeding, as if every write had been a no-op.

The root cause was in `nova_div` in `nova_rt.c`. The function always returned `TAG_FLOAT`, even for two integer operands, because the original rationale for Nova’s division operator is that it always produces a float ($10 / 3 = 3.333\dots$). This was correct for the interactive backends.

In the GoL parallel implementation, the chunk size is computed as:

$$\text{chunk} = h / \text{nthreads} = 512 / 4 = 128$$

With `nova_div` returning float, `chunk` had `TAG_FLOAT` with value 128.0. The start and end indices derived from it were then also floats. Inside `nova_index_set` in `nova_rt.c`, the index bounds check requires `idx->tag == TAG_INT` before performing the write. A float index fails this check and the function returns without writing. Every cell write in every worker was therefore silently discarded.

The fix adds an integer fast path to `nova_div`:

```
1 void nova_div(const NovaValue* a, const NovaValue* b, NovaValue* o) {
2     if (a->tag == TAG_INT && b->tag == TAG_INT) {
3         int64_t bv = (int64_t)b->payload;
4         if (bv == 0) { nova_throw_error("div: division by zero"); return; }
5         o->tag = TAG_INT;
6         o->payload = (uint64_t)((int64_t)a->payload / bv);
7         return;
8     }
9     /* float path unchanged */
10 }
```

Listing 12: Integer fast path added to `nova_div`

Integer/integer division now returns `TAG_INT` via truncating division, consistent with Go, C, and Rust. Float operands still produce float output.

4.6 Concurrency Runtime

The concurrency runtime is implemented across `evaluator.rs`, `vm.rs`, and `nova_rt.c` depending on the backend.

Thread creation. The tree-walker and VM spawn OS threads directly via `std::thread::spawn` with rayon’s thread pool for load balancing. The LLVM backend uses `Win32 CreateThread` on Windows and `POSIX pthread_create` on Linux and macOS, implemented in `nova_rt.c` with a compile-time `#ifdef _WIN32` switch.

Channels. The tree-walker and VM use crossbeam’s unbounded MPMC channel internally. Each `Value::Channel` wraps an `Arc<ChannelInner>` that holds the sender/receiver pair and a `pending: Mutex<Vec<Value>>` mirror of in-flight values. The pending list exists solely for the garbage collector (GC): without it, the GC cannot see values that have been sent but not yet received, and would clear them. The LLVM backend uses a condition variable and mutex in C for the same semantics.

Mutex as semaphore. Nova’s `mutex` is implemented as a bounded(1) crossbeam channel. The channel’s single-slot capacity enforces mutual exclusion: only one task can hold the token at any time. This reuses the existing channel infrastructure with no new Rust synchronisation primitives.

Deep-clone on spawn. When a task is spawned, its arguments are processed by `deep_clone_unregistered`: a recursive function that creates a fully independent copy of the value graph without registering the new allocations in the GC heap. Unregistered clones are necessary to prevent the parent’s GC from freeing data that belongs to the spawned thread, a bug that manifested as empty arrays in the first implementation of `spawn`.

5 Evaluation

5.1 Test Suite and Correctness Methodology

Nova’s correctness methodology is differential testing: the same `.nova` file is run on all available backends, and their outputs are compared line by line. A discrepancy between the tree-walker and the VM or LLVM backend indicates a bug in the newer backend rather than in the test expectation. This approach was applied throughout development: every new feature was validated by running the same program on the tree-walker (reference) and the VM or LLVM backend (under test) and confirming identical output.

The test suite comprises approximately 45 test files in `testFiles/`, covering:

- Core language: closures, error handling, edge cases, patterns, pipes (`test_closures.nova`, `test_errors.nova`, etc.)
- Additional language features: bitwise operators, structs, enums, methods, for-loop destructuring, default parameters, function types
- Stress tests: scoping edge cases, arithmetic edge cases, deep recursion, large collections, HOF chains, concurrency primitives
- Correctness under load: `stress.nova` (400 lines, 15 sections, checks the combined behaviour of all language features)
- Numeric edge cases: infinity, NaN, large integers, modulo semantics
- Concurrency: channels, spawn/wait, spawnAll, select, defer, mutex

Early in development, a grammar-based fuzzer generated 200 random valid Nova programs and ran them through the tree-walker. Zero panics were found. A mutation fuzzer applied 18 mutation types (operator swaps, value replacements, line deletions) to all existing test files. After fixing two parser bounds issues found in the first run, a second pass produced zero crashes and only logically correct infinite loops (deleted `break` statements).

5.2 VM vs Tree-Walker Performance

All benchmark timings were recorded using Nova’s built-in `clock()` function, which returns wall-clock milliseconds since the Unix epoch as an integer. Elapsed time is computed as `clock()` called before and after the workload, with the difference printed as the result. No external profiler or OS timer was used.

Two benchmarks measured the performance gap between the bytecode VM and the tree-walker. Both were run as debug builds on an Intel Core i7-11370H @ 3.30 GHz, Windows 11.

The first benchmark (`benchmark.nova`) runs five workloads back-to-back as a single timed script: `fib(28)`, a 100 000-iteration loop, sort/filter/map on a 1 000-element array, 1 000 string operations, and 500 hashmap operations.

Benchmark	Tree-walker	VM	Speedup
<code>benchmark.nova</code> (5 workloads combined)	11.9 s	3.8 s	$\approx 3.1\times$

Table 1: VM vs tree-walker performance (`benchmark.nova`), debug build

After these measurements were taken, the LLVM runtime library was extended to support higher-order functions (`map`, `filter`), `sum`, and hashmap operations, making `benchmark.nova` runnable on all three backends. Table 2 shows a

subsequent three-way comparison measured with PowerShell `Measure-Command` on the same machine (release build). The LLVM time includes the per-run `clang` compilation step; a pre-compiled binary would be faster for repeated runs.

Backend	Time	Speedup vs VM
Tree-walker	11.9 s	0.7×
VM	5.1 s	1.0× (baseline)
LLVM	1.0 s	≈5×

Table 2: `benchmark.nova` across all three backends (release build, `Measure-Command`)

Metric	Tree-walker	VM
Total time (6 sections)	36.5 s	25.5 s (≈1.4× faster)
Recursion depth	500 (stack overflow)	Unlimited

Table 3: VM vs tree-walker performance (`bench_vm.nova`)

The VM is approximately 3.1× faster on Higher-Order Function (HOF) and string-heavy workloads, and 1.4× faster on pure-computation workloads. The speedup on HOF workloads is larger because the VM eliminates the overhead of HashMap cloning on every function call, which dominates the tree-walker’s cost for `map/filter` chains that call a lambda thousands of times. The VM also removes the recursion depth limit entirely: `rsum(1000)` (a recursive sum that requires 1001 frames) crashes the tree-walker and completes normally on the VM.

The LLVM backend achieves a further ≈5× speedup over the VM on the same workload. The gain comes from two sources: native code generation eliminates the bytecode dispatch loop, and the type specialisation pass replaces generic tagged-value arithmetic with direct LLVM integer and floating-point instructions for statically-known types. The HOF sections (`map`, `filter`) call closures through the C runtime rather than an inlined VM loop, which partially offsets the gain in those sections.

5.3 Conway’s Game of Life: Serial vs Parallel

Conway’s Game of Life (GoL) was used as the primary benchmark for the LLVM backend and the concurrency runtime. The benchmark uses a 512×512 toroidal grid, 1000 generations, and 4 worker threads.

The parallel implementation divides the grid into horizontal strips, one per thread. Each worker is responsible for a contiguous range of rows and writes its results into a pre-allocated output buffer. The main thread spawns all workers, waits for all handles, then swaps the buffers for the next generation. This double-buffer pattern avoids allocating a new grid each generation.

```

1 fn next_gen_parallel_into(grid, next, w, h, nthreads) {
2     let chunk = h / nthreads
3     let handles = []
4     for t in 0..nthreads {
5         let start = t * chunk
6         let end = if t == nthreads - 1 { h } else { start + chunk }
7         let handle = spawn worker_fill(grid, next, w, h, start, end)
8         handles = push(handles, handle)
9     }
10    for handle in handles { wait(handle) }

```

Listing 13: Parallel GoL generation step in Nova

Mode	Time (ms)	Alive count	Notes
Serial	43,160	6,135	
Parallel (4 threads)	21,319	6,135	$2.02\times$ speedup

Table 4: Nova GoL benchmark results (512×512, 1000 generations, Intel Core i7-11370H, Windows 11)

Both modes produce identical alive counts, confirming correctness of the parallel implementation after the integer division bug was fixed.

5.4 Nova vs Go: Parallel Performance Comparison

The Go comparison uses an equivalent benchmark written as part of a separate university project. The Go implementation uses the same algorithm (8-neighbour wrap-around with bitwise modulo, double buffering) and the same parallelisation strategy (one goroutine per strip). The data type is `[] []byte` (1 byte per cell) compared to Nova’s `NovaValue` array (16 bytes per cell).

Mode	Time (ms)	Notes
Serial	1,726	<code>go test -bench=BenchmarkSerial -benchtime=3x</code>
Parallel (4 goroutines)	797	<code>go test -bench=BenchmarkParallel4 -benchtime=3x</code>
Speedup	$2.17\times$	

Table 5: Go GoL benchmark results (512×512, 1000 generations, same hardware)

Metric	Nova (LLVM)	Go
Serial time	43,160 ms	1,726 ms
Parallel time (4 threads)	21,319 ms	797 ms
Parallel speedup	$2.02\times$	$2.17\times$
Parallel efficiency (of 4×)	50.5%	54.3%
Raw gap (serial)	$25\times$ slower	—
Raw gap (parallel)	$27\times$ slower	—

Table 6: Nova vs Go parallel efficiency comparison

Raw performance gap. Nova’s serial time is approximately $25\times$ slower than Go’s. This is expected and fully explained by representational differences. Go stores each cell as a `byte` (1 byte); Nova stores each cell as a `NovaValue` struct (16 bytes). The 512×512 grid occupies 256 KB in Go (fits in L2 cache) and approximately 4 MB in Nova (does not fit). Furthermore, every array read and write in Nova goes through `nova_index_get` and `nova_index_set`, full C function calls with tag dispatch and bounds checking, whereas Go’s slice indexing compiles to a single load or store instruction with bounds checks typically eliminated by the compiler. The $25\times$ gap is the expected cost of a dynamically-typed language using a tagged runtime representation on a memory-bandwidth-bound workload.

Parallel efficiency. The parallel efficiencies are nearly identical: 50.5% for Nova and 54.3% for Go. Both are limited by per-generation thread creation overhead. With 1000 generations and 4 workers, 4000 thread creation cycles occur per run. Nova uses Win32 `CreateThread` (OS thread, approximately 50–200 μ s creation cost); Go uses goroutines (lightweight green threads, approximately 1–5 μ s creation cost). Despite Go’s goroutines being approximately 50 $\times$ cheaper to create, the parallel efficiency gap is only 4 percentage points. This is because the ratio of thread-creation overhead to per-generation compute work is similar for both languages at this grid size: Nova’s heavier per-cell cost offsets the heavier thread creation cost.

Key finding. Nova’s `spawn/wait` parallelism model achieves the same parallel efficiency as Go’s goroutines for this workload class. The absolute performance difference is entirely attributable to the dynamic-typed runtime representation, not to any deficiency in the concurrency implementation.

6 Discussion

6.1 Key Design Decisions and Trade-offs

Three backends. Building three backends in order of increasing complexity (tree-walker $\rightarrow$ VM $\rightarrow$ LLVM) served a dual purpose. Each backend provided a working implementation to test the next one against (differential testing), and each revealed a new class of implementation challenge: pointer tree performance for the tree-walker, borrow-checker conflicts for the VM, and SSA form and ABI issues for LLVM. The alternative, going directly to LLVM from the tree-walker, would have removed the VM as a correctness reference and made debugging the LLVM backend much harder.

Copy-on-write for arrays. CoW was chosen over deep-copy-on-assign (simple but $O(n)$ per assignment) and reference semantics ($O(1)$ but aliasing surprises). CoW gives value semantics with near-reference performance: reads are always $O(1)$ (no copy), and writes only pay a copy cost when the data is actually shared. An unexpected benefit was that CoW eliminated reference cycles as a side effect: any self-referential assignment first triggers a copy (because the in-progress evaluation briefly increments the strong count), so the resulting structure can never contain a true cycle. The mark-and-sweep collector therefore always reports zero cycles collected in practice.

Gradual typing over strict typing. Enforcing types on every function would have broken all existing Nova code and made the language significantly less accessible. Gradual typing means developers can add annotations incrementally, and the checker only fires when it has enough information to be certain of a violation. The three-pass design ensures that forward references work and that unannotated functions are still checked when their types can be inferred, without requiring the developer to annotate every function before any checking is done.

Deep-clone on spawn. The most common class of Go concurrency bug is a closure that captures a slice reference and races with the parent. Nova prevents this by deep-cloning all spawn arguments. The cost is $O(n)$ in the size of the argument, but for the common case (spawning a task with a small configuration object or a data chunk for processing), this is dominated by the thread creation cost. The benefit is that data races of this type are impossible by construction, without requiring the developer to reason about which variables are shared.

nova_rt.c as a C library. Writing the runtime in C rather than in Rust or directly in LLVM IR kept the code generator simple (pure string concatenation), kept the runtime readable and debuggable, and made it straightforward to add new operations (adding a function to `nova_rt.c` requires no changes to the IR schema). The cost is that all dynamic operations are C function calls rather than inline IR, which limits the LLVM optimiser’s ability to inline and specialise them. The type specialisation pass (Section 4.5) mitigates this for statically-known types.

6.2 What Was Harder Than Expected

The Rust borrow checker in the VM. The VM’s main loop reads the current instruction from `frames.last()` (an immutable borrow) and then needs to mutate `frames` (for `Call` and `Return`). Rust does not allow an immutable borrow and a mutable borrow of the same vector to coexist. The solution, cloning the instruction immediately after reading it and dropping the borrow before any mutation, is simple once found but took significant debugging to identify. This pattern appears throughout the VM dispatch loop.

is_value_expr and match-as-expression. The VM compiler has a flag, `is_value_expr(stmt)`, that decides whether the last statement in a function body should be treated as an expression (leaving its result on the stack as the return value) or as a statement (discarding the result). Adding the `match` construct required adding it to this list; forgetting to do so caused functions whose entire body was a `match` expression to silently return `nil`. The bug was non-obvious because the symptom (wrong return value) looked identical to a logic error in the match arms.

The alloca hoisting pass. The connection between stack-allocated temporaries in non-entry LLVM IR blocks and the resulting `sub rsp, 16` on every loop iteration was not immediately apparent. The symptom, a silent process crash with no output, gave no indication that the problem was stack growth rather than a logic error. Diagnosing it required understanding LLVM’s `mem2reg` pass and its entry-block requirement, then writing a post-processing pass over the raw IR text.

The integer division type bug. The parallel workers executing without errors but producing no writes was a difficult failure mode. The root cause (a function returning the wrong tag for its output) was three levels of indirection from the symptom (wrong alive count): wrong tag $\rightarrow$ float indices $\rightarrow$ `nova_index_set` silent no-op $\rightarrow$ alive count unchanged. The debugging process required writing a series of minimum reproducers to isolate each layer of the causal chain.

Arc migration. Adding concurrency required migrating all heap-allocated values from `Rc<RefCell<T>>` (single-threaded) to `Arc<Mutex<T>>` (thread-safe). Because `Rc` and `Arc` have identical APIs, a bulk find-replace was attempted. Several sites were missed (CoW paths, pointer equality checks) that only manifested as compile errors. The lesson is that large type-system migrations benefit from a systematic search for every site rather than a text replacement.

6.3 Limitations

Raw performance. Nova is approximately $25\times$ slower than Go on computation-heavy workloads. The primary contributor is the `NovaValue` tagged union (16 bytes), which makes the grid $16\times$ larger than Go’s native byte array, causing L2 cache thrashing. Type specialisation partially addresses this for scalar integer arithmetic but does not help with array accesses, which still go through `nova_index_get` and `nova_index_set`.

LLVM backend memory. The LLVM backend frees all tracked memory at program exit via `nova_gc()`. There is no runtime garbage collection during execution. Long-running programs grow their heap monotonically. For the GoL benchmark (which exits after 1000 generations), this is acceptable; for a server or daemon, it would be a leak.

Concurrency safety gap vs Erlang. The deep-clone on spawn prevents closure capture races but does not provide full process isolation. Shared state is still possible through mutexes and channels. A crash in a spawned task is not automatically restarted by a supervisor. Nova’s concurrency is meaningfully safer than Go by default, but the distance to Erlang’s structural fault tolerance is substantial.

“Errors that fix themselves.” The original third pillar of Nova’s design, a runtime that observes failures and proposes or applies fixes, was not implemented. The type checker provides “did you mean X?” suggestions via Damerau-Levenshtein distance for undefined variables, but runtime errors do not query an AI model for suggestions. This remains entirely future work.

No standard library or package manager. Nova has approximately 60 built-in functions but no module system beyond `import "path.nova"`. There is no HTTP client, JSON parser, date/time support, or package registry. Without these, Nova programs cannot interact with the world beyond files and the terminal.

7 Conclusion

7.1 Summary of Contributions

This project designed and implemented a general-purpose programming language from first principles in Rust. The key contributions are:

1. A complete front-end pipeline (lexer, parser, warnings checker, gradual static type checker with inference and generics) shared across all three backends.
2. A tree-walking interpreter, a bytecode compiler and virtual machine (1.4–3× faster than the tree-walker), and an LLVM native code generator, three backends representing three distinct points on the performance/complexity spectrum.
3. A hand-written C runtime library (`nova_rt.c`) implementing all dynamic operations for the LLVM backend, with a tracked allocation pool and full cleanup at program exit.
4. A post-processing LLVM IR pass (`hoist_allocas`) that moves all `NovaValue` allocations to function entry blocks, enabling `mem2reg` promotion and preventing stack growth in loops.
5. A concurrent runtime with `spawn/wait`, typed channels, mutex, `select`, `defer`, and timers, with share-nothing semantics enforced by deep-clone on `spawn`.
6. A Conway’s Game of Life benchmark demonstrating 2.02× parallel speedup with Nova and 2.17× with Go, nearly identical parallel efficiency despite a 25× gap in raw throughput.

7.2 Lessons Learned

Start with the simplest correct thing. The tree-walker was the right first backend. Its correctness was easy to verify, and it provided the reference implementation that made all subsequent work testable. Skipping it in favour of the VM or LLVM directly would have removed the differential testing foundation.

Differential testing is a highly effective correctness strategy. Running the same program on two independent backends and comparing output caught every significant bug in the VM and LLVM backends. Many of these bugs, such as the `is_value_expr` match-expression bug, were silent wrong-value errors that would have been invisible to any test that only checked for crashes or exceptions.

LLVM IR text is easier to work with than the C++ API. Generating LLVM IR as a string, rather than through the `inkwell` Rust bindings for the LLVM C++ API, kept the code generator simple and made incorrect output immediately readable in a text editor. The cost is that the IR is not validated by the LLVM library, but in practice the compiler caught most structural errors.

Concurrency makes everything harder. Migrating from `Rc<RefCell<T>>` to `Arc<Mutex<T>>`, ensuring deep-clone semantics for `spawn`, handling errors from spawned threads, and preventing the GC from freeing data owned by running threads each required non-trivial changes throughout the codebase. Concurrency should be designed into the memory model from the beginning, not added later.

7.3 Future Directions

Supervision trees. The most important missing feature for the language’s original safety goals. Tasks that crash should be automatically restarted by a supervisor. A `supervise { spawn worker() restart: always }` syntax would bring Nova meaningfully closer to Erlang-style structural fault tolerance.

Standard library and package manager. Without HTTP, JSON, and a module registry, Nova programs cannot interact with real-world systems. A minimal standard library and a simple package manager are the prerequisites for any practical adoption.

Improved runtime performance. The $25\times$ gap to Go could be reduced by several means: extending type specialisation to array accesses (so `arr[i]` on a statically-typed integer array compiles to a direct LLVM `load`), replacing per-generation thread creation with a persistent thread pool, and adding a reference-counted or arena-based memory model to the LLVM backend.

AI-assisted error messages. The minimum viable version of the original “errors that fix themselves” pillar: when a compile-time or runtime error fires, query a language model with the source context and error message and append a suggested fix to the error output. This would be a genuine differentiator from existing languages and is achievable with the existing Nova infrastructure.

References

Lattner, C. and Adve, V. (2004) ‘LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation’, *Proceedings of the 2004 International Symposium on Code Generation and Optimization (CGO’04)*, Palo Alto, California, March 2004.

Nystrom, R. (2021) *Crafting Interpreters*. Genever Benning. Available at: <https://craftinginterpreters.com>.

Siek, J.G. and Taha, W. (2006) ‘Gradual Typing for Functional Languages’, *Proceedings of the 2006 Scheme and Functional Programming Workshop*, University of Chicago Technical Report TR-2006-06, pp. 81–92.